2026

Stage 1– Bulding the Data Layer.

LAURA MEDINA ARENCIBIA – 45179662A
DANIELA ERIDENIA MARTEL VALIDO – 54164703W
FRANCISCO JAVIER GUERRA REDONDO – 49979487G
ALEJANDRO MORALES MONROY – 46248675E

BIG DATA GROUP.
URL REPOSITORY: HTTPS://GITHUB.COM/BIG-DATAGROUP/STAGE_1.GIT

Index

Introduction and objetives. 2
System Architecture. 4
Datalake 4
Datamarts 5
Control Layer 5
Design Decisions. 6
Datalake Structure 6
Datamart Organization 7
Control Layer 8
Benchmarks and results. 9
Methodology 9
Results by Language 10
C# 10
Java 10
Python 11
Cross-Language Comparison 12
Comparison of Index Structures 12
Conclusions and future improvements. 14

Introduction and objetives.

The project developed in the Big Data course aims to build a search engine from scratch, capable of downloading, processing, storing, and indexing large volumes of textual information obtained from Project Gutenberg. The system replicates the core architecture of modern search engines, implementing data ingestion pipelines, scalable storage systems, and efficient indexing structures.

Stage 1 serves as the foundation of the entire project. In this phase, the data layer is designed and implemented, providing the essential infrastructure upon which the later stages (crawling, indexing, and querying). Its role within the complete system is to ensure that data is ingested in an organized manner, properly cleaned, and stored in structures that support efficient retrieval and processing. Without a robust data layer, the search engine would not be able to operate reliably or efficiently.

The main objectives of Stage 1 are the following:

1. Construction of the datalake.

Build a scalable and well-organized repository to store both raw and cleaned book content. The datalake must support traceability, incremental processing, and benchmarking of different storage structures.

2. Creation of the datamarts.

Transform the cleaned data into optimized, queryable structures. This includes:

- Extracting and storing book metadata in a structured database.
- Building multiple versions of the inverted index to compare performance and understand how different designs affect search and update efficiency.

3. Establishment of the control layer.

Implement a simple orchestration mechanism that tracks the state of the pipeline, prevents duplicated work, and coordinates the basic tasks of downloading and indexing books. This component ensures that the workflow remains reliable and consistent.

Together, these objectives form the backbone of the search engine, preparing all necessary data and structures so that subsequent stages can focus on advanced indexing, query execution, and full system integration.

System Architecture.

The system architecture developed in Stage 1 defines how data is organized and flows through the search engine. This phase focuses on building the data layer, which serves as the foundation for the indexing and querying stages that follow. The architecture is composed of three main components: the datalake, the datamarts, and the control layer. Each component plays a specific role within the pipeline, ensuring an organized, scalable, and traceable workflow.

Datalake.

The datalake is the central repository where both raw and cleaned book content from Project Gutenberg is stored. Its design must be scalable and allow clear organization of data.

In this project, the datalake is structured using a time-based hierarchy, organizing files according to the date and hour of ingestion. This approach provides:

- Full traceability of each downloaded book.
- Efficient detection of new data for incremental processing.
- Balanced distribution of files to avoid overloading directories.

Each book is split into two parts: header (metadata) and body (book content), which are stored as separate files within the datalake.

Datamarts.

The datamarts transform cleaned data into optimized, queryable structures. They consist of two components:

- **Metadata**:

  Metadata extracted from the header is stored in a structured database (e.g., SQLite). This enables fast queries such as filtering books by author, title, or language, and allows quick access to the corresponding text stored in the datalake.

- **Inverted Index**:

  The body of each book is used to build multiple versions of the inverted index. This index maps each term to the books in which it appears, enabling efficient search operations.

Several alternative index structures (monolithic file, MongoDB, hierarchical folder structure) are implemented and compared in terms of performance, scalability, and update efficiency.

Control Layer.

The control layer acts as the coordinator of the pipeline. Its purpose is to track the system's state and prevent duplicated work. It uses control files to record:

- Downloaded books.
- Indexed books.

This component decides which action to perform at each step: download a new book or index a pending one. It ensures a reliable workflow where each book is processed exactly once.

Design Decisions.

Stage 1 of the project requires several design decisions that directly impact the performance, scalability, and organization of the system. These decisions focus on three main areas: the datalake structure, the datamart organization, and the implementation of the inverted index. The following sections describe and justify the choices made.

Datalake Structure.

To store the books downloaded from Project Gutenberg, different directory structures were evaluated. The final choice was a time-based hierarchy.

Justification:

- Provides clear traceability of ingestion time.
- Enables efficient incremental processing by easily identifying new data.
- Prevents excessive accumulation of files in a single directory.
- Reflects common practices in distributed storage systems such as HDFS or S3.

Although alternatives such as book-based or range-based structures were considered, the time-based layout offered the best balance between simplicity and scalability.

Datamart Organization.

The datamarts were designed to transform cleaned data into optimized, queryable structures. Two key decisions were made:

Metadata:

Metadata extracted from the header is stored in a structured database.

Reasons:

- Enables fast queries by author, title, or language.
- Simplifies locating the corresponding text in the datalake.
- Lightweight, portable, and adequate for the project's data volume.

Alternatives such as PostgreSQL or MongoDB were considered, but SQLite provided the best balance between simplicity and functionality for this stage.

Inverted Index:

Three different index structures were implemented to compare performance:

- Single monolithic JSON file.
- MongoDB.
- Hierarchical folder structure with one file per term.

Reasons for this selection:

- Covers centralized, distributed, and fine-grained approaches.
- Allows comparison of indexing, querying, and update performance.
- Reflects real-world strategies used in search systems.

Control Layer.

The control layer was designed as a lightweight component that coordinates the pipeline using simple state files.

Justification:

- Easy to implement and sufficient to prevent duplicated work.
- Makes it straightforward to identify books pending indexing.
- Integrates well with later stages without adding unnecessary complexity.

Conclusion

The design decisions made in this stage aim to balance simplicity, performance, and scalability. Each choice contributes to building a robust data layer that supports the indexing and querying phases of the search engine.

Benchmarks and results.

This section presents a comparative performance analysis of the system using three programming languages (C#, Java, and Python) and three different storage strategies for both the datalake and the inverted index. The goal is to evaluate how design decisions affect performance in key operations such as index updates, query execution, and disk I/O.

Methodology.

Three datalake structures were evaluated:

- by_book → each book stored in its own folder.
- by_batch → books grouped in ranges of 1000.
- by_time → organization by ingestion date and hour.

Three inverted index structures were also implemented:

- Monolithic JSON file.
- MongoDB (NoSQL).
- Hierarchical folder-based index.

Benchmarks were executed using datasets of 500 and 1000 books.

Three core operations were measured:

- updatePerformance → cost of updating the inverted index.
- queryPerformance → speed of search queries.
- diskIoPerformance → efficiency of disk read/write operations.

Results by Language.

C#.

C# exhibited extremely low execution times in nanoseconds, with zero memory allocations, indicating highly optimized JIT compilation and efficient in-memory index handling.

Operation	Dataset	Mean (ns)	StdDev (ns)
UpdatePerformance	500	0.0006	0.0013
QueryPerformance	500	0.0077	0.0126
DiskIoPerformance	500	0.0095	0.0121
UpdatePerformance	1000	0.0150	0.0176
QueryPerformance	1000	0.0095	0.0101
DiskIoPerformance	1000	0.0134	0.0231

C# Conclusion:

Outstanding CPU performance and extremely efficient memory usage. Ideal for high-frequency index updates and low-latency operations.

Java.

Java demonstrated very high throughput, especially in query execution and disk I/O. Garbage collection activity was present but controlled.

Disk I/O:

Operation	Dataset	Throughput (ops/s)	GC Alloc (B/op)	GC Count	GC Time (ms)
diskIoPerformance	500	1,188,997,080.51	0.000006	0	NaN
diskIoPerformance	1000	1,185,590,185.98	0.000006	0	NaN

Query Performance:

Operation	Dataset	Throughput (ops/s)	GC Alloc (B/op)	GC Count	GC Time (ms)
queryPerformance	500	140,879,872.11	24.000049	56	49
queryPerformance	1000	140,616,955.16	24.000050	54	51

Update Performance:

Operation	Dataset	Throughput (ops/s)	GC Alloc (B/op)	GC Count	GC Time (ms)
updatePerformance	500	1,230.84	1,387,274.87	37	28
updatePerformance	1000	628.03	2,747,819.14	37	31

Java Conclusion:

Excellent throughput in read-heavy workloads. GC introduces overhead in update operations, but overall performance remains strong and stable.

Python.

Python delivered modest performance, as expected from an interpreted language, but still achieved competitive results in query and disk operations.

Operation	Dataset	Ops/s	RAM (MB)
UpdatePerformance	500	6.53	0.0559
QueryPerformance	500	142,857	0.0001
DiskIoPerformance	500	222,222	0.0001
UpdatePerformance	1000	3.34	0.0555
QueryPerformance	1000	131,578	0.0001
DiskIoPerformance	1000	196,078	0.0001

Python Conclusion:

Suitable for prototyping and analysis. Query and disk operations perform well, but index updates are significantly slower due to Python's overhead in handling large in-memory structures.

Cross-Language Comparison.

Operation	Best Language	Reason
UpdatePerformance	C#	Minimal execution time and zero allocations
QueryPerformance	Java	Extremely high throughput and efficient GC
DiskIoPerformance	Java	Fastest disk read/write operations
Memory Efficiency	C#	No memory allocations in any benchmark

Comparison of Index Structures.

Monolithic JSON.

- Fast sequential reads.
- Slow for large updates.
- Best for simple search workloads.

MongoDB (NoSQL).

- Excellent incremental update performance.
- Highly scalable.
- Slightly higher latency for simple queries.

Hierarchical Folder Structure.

- Very fast updates (only one file modified per term).
- Slower queries due to multiple file accesses.
- Best for debugging and fine-grained control.

Conclusions and future improvements.

The performance analysis presented in the previous section reveals clear differences between the three evaluated languages (C#, Java, and Python) and the various datalake and inverted index structures. Although each language offers specific advantages, the overall results indicate that Java provides the best balance of performance, stability, and scalability, particularly for workloads involving large datasets.

Java stands out due to its high throughput in queries and disk operations, consistent GC behavior, and a mature ecosystem that supports advanced benchmarking, concurrency libraries, and storage systems. It also integrates naturally with all three implemented inverted index structures (monolithic, MongoDB, and hierarchical), enabling flexible architectural decisions as the project evolves.

For these reasons, Java is recommended as the primary language for future development of the search engine, especially in stages where scalability, robustness, and real-world workload handling will be essential.